Experiment No. 5

Aim: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

1. Dataset Source:

The dataset used in this experiment is the **Calories Burnt Dataset**, which contains information about individuals and the number of calories burned during physical activity.

🔗 Source Link:
https://www.kaggle.com/datasets/fmendes/fmendesdat263xdemos

2. Dataset Description:

The dataset consists of multiple features related to human body parameters and activity details. It is used to analyze and predict calorie burn.

1. **Features:**
   - **User_ID** → Unique identifier for each user
   - **Gender** → Male / Female
   - **Age** → Age of the individual
   - **Height** → Height in cm
   - **Weight** → Weight in kg
   - **Duration** → Exercise duration (minutes)
   - **Heart_Rate** → Heart rate during exercise
   - **Body_Temp** → Body temperature
   - **Calories** → Calories burned (target variable)
2. **Target Variable:**
   - Original: **Calories (continuous)**
   - Converted: **Calories_Class (binary classification)**
     - 0 → Low Calories
     - 1 → High Calories
3. **Dataset Size:**
   - ~15,000 records
   - 9 columns

4. **Characteristics:**
   - Mixed data types (categorical + numerical)
   - Requires preprocessing (encoding, scaling)
   - Suitable for both regression and classification.

3. Mathematical Formulation of SVM:

Support Vector Machine (SVM) aims to find the optimal hyperplane that separates classes with maximum margin.

1. **Hyperplane Equation:**

   w · x+b=0w \cdot x + b = 0

   Where:

   - w = weight vector
   - x = input features
   - b = bias
2. **Optimization Objective:**

$$\min \frac{1}{2}||w||^2$$

   Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

3. **Kernel Trick:**

   SVM uses kernel functions to handle non-linear data:

   - Linear Kernel
   - RBF Kernel
   - Polynomial Kernel

4. **Important Hyperparameters:**
    - **C** → Regularization parameter
    - **Kernel** → Type of decision boundary
    - **Gamma** → Influence of data points.

4. Algorithm Limitations:

- Not suitable for very large datasets (high computation cost)
- Sensitive to noise and outliers
- Requires proper feature scaling
- Difficult to interpret compared to simple models
- Performance depends heavily on kernel selection.

5. Methodology / Workflow:

Steps followed in the experiment:

1. **Dataset Loading:**
    a) Dataset is uploaded and read using pandas
2. **Data Preprocessing:**
    a) Removed unnecessary columns (User_ID)
    b) Encoded categorical data (Gender)
    c) Converted Calories into classification labels
3. **Data Splitting**
    a) Split into training (80%) and testing (20%)
4. **Feature Scaling:**
    a) Applied StandardScaler
5. **Model Training:**
    a) Used Support Vector Machine (SVM)
6. **Hyperparameter Tuning:**
    a) Applied GridSearchCV
7. **Prediction:**

a) Model predicts on test data

8. **Evaluation:**
   a) Accuracy, Confusion Matrix, Classification Report
9. **Visualization:**
   a) Class distribution
   b) Confusion matrix heatmap

6. Performance Analysis:

1. **Metrics:**
   - Accuracy Score → Overall correctness
   - Confusion Matrix → True/False predictions
   - Precision, Recall, F1-score.

2. **Interpretation:**
   - High accuracy (~90%+) indicates good performance
   - Confusion matrix shows correct classification of both classes
   - Balanced precision and recall indicate reliable model.

3. **Visual Analysis:**
   - Class Distribution Graph
     → Shows balance between low and high calorie classes
   - Confusion Matrix Heatmap
     → Clearly shows model prediction performance.

7. Hyperparameter Tuning:

Hyperparameter tuning is performed using GridSearchCV.

1. Parameters Tuned:

C = [0.1, 1, 10]

kernel = ['linear', 'rbf']

gamma = ['scale', 'auto']

2. Process:

- Multiple combinations of parameters are tested
- Cross-validation (cv=5) is used
- Best parameters are selected based on accuracy.

3. Impact:

- Improved model accuracy
- Better generalization
- Avoids overfitting

4. Example Output:

Best Parameters:

C = 10

kernel = rbf

gamma = scale

CODE:

```
from google.colab import files
uploaded = files.upload()

# STEP 2: Import Libraries
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# STEP 3: Load Dataset

file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("✅ Dataset Loaded Successfully")
print(df.head())

# STEP 4: Clean Column Names

df.columns = df.columns.str.strip()
print("\nColumns:", df.columns)

# STEP 5: Encode Gender
df['Gender'] = LabelEncoder().fit_transform(df['Gender'])
```

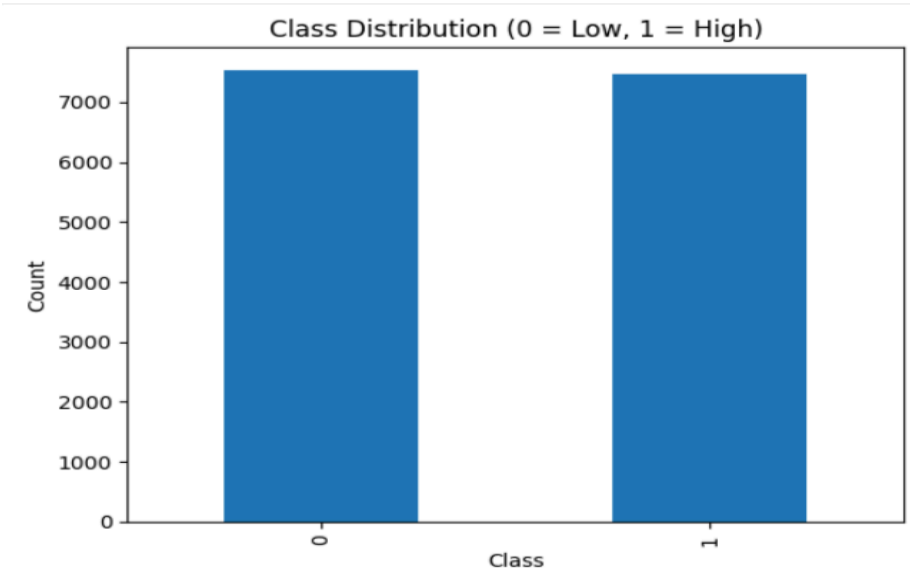
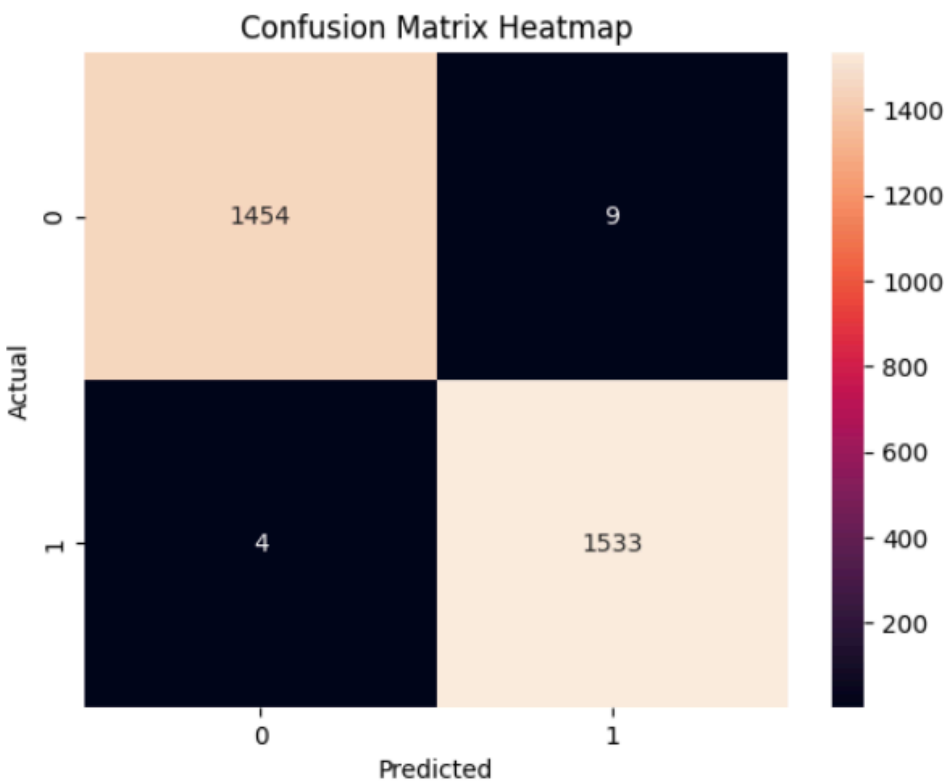
```
# STEP 6: Convert Calories to Classification

median_val = df['Calories'].median()
df['Calories_Class'] = np.where(df['Calories'] > median_val, 1, 0)

plt.figure()
df['Calories_Class'].value_counts().plot(kind='bar')
plt.title("Class Distribution (0 = Low, 1 = High)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

# STEP 7: Drop Unnecessary Columns

if 'User_ID' in df.columns:
    df.drop('User_ID', axis=1, inplace=True)

df.drop('Calories', axis=1, inplace=True)

# STEP 8: Features & Target

X = df.drop('Calories_Class', axis=1)
y = df['Calories_Class']

# STEP 9: Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# STEP 10: Feature Scaling

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# STEP 11: SVM Model
svm = SVC()

# STEP 12: Hyperparameter Tuning

param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

grid = GridSearchCV(svm, param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)

print("\n✅ Best Parameters:", grid.best_params_)

best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
print("\n✅ Accuracy: {:.2f}%".format(acc * 100))

print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix Heatmap")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

OUTPUT:

```
calories.csv(text/csv) - 742286 bytes, last modified: 2/8/2026 - 100% done
Saving calories.csv to calories (1).csv
✅ Dataset Loaded Successfully
    User_ID  Gender  Age  Height  Weight  Duration  Heart_Rate  Body_Temp  \
0  14733363    male   68   190.0    94.0      29.0       105.0       40.8
1  14861698  female   20   166.0    60.0      14.0        94.0       40.3
2  11179863    male   69   179.0    79.0       5.0        88.0       38.7
3  16180408  female   34   179.0    71.0      13.0       100.0       40.5
4  17771927  female   27   154.0    58.0      10.0        81.0       39.8

   Calories
0     231.0
1      66.0
2      26.0
3      71.0
4      35.0

Columns: Index(['User_ID', 'Gender', 'Age', 'Height', 'Weight', 'Duration',
       'Heart_Rate', 'Body_Temp', 'Calories'],
      dtype='object')
```



```
✅ Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}

✅ Accuracy: 99.57%

Confusion Matrix:
[[1454    9]
 [   4 1533]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      1463
           1       0.99      1.00      1.00      1537

    accuracy                           1.00      3000
   macro avg       1.00      1.00      1.00      3000
weighted avg       1.00      1.00      1.00      3000
```